

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Nirmal Kasi	Reg. No.:	192424248
Section / Batch:	B.Tech AI/DS	Date:	

Code of Conduct

I Nirmal Kasi 192424248 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nirmal

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Q1. Error Analysis of Morphological Decomposition

1. Correct Morphological Analysis

Word	Decomposition	Type
treatment	treat + ment	-ment is derivational
treatable	treat + able	-able is derivational
retreatment	re + treat + ment	re- and -ment are derivational
treated	treat + ed	-ed is inflectional
untreated	un + treat + ed	un- is derivational, -ed is inflectional

2. Explanation

Treatment

treat + ment

-ment changes the verb `treat` into the noun `treatment`, so it is **derivational**.

Treatable

treat + able

-able forms an adjective meaning "can be treated", so it is **derivational**.

Retreatment

re + treat + ment

re- adds the meaning "again", while -ment forms a noun. Both are **derivational**.

Treated

treat + ed

-ed indicates past tense, so it is **inflectional**.

Untreated

un + treat + ed

un- changes the meaning to "not treated" and is **derivational**. -ed indicates past tense and is **inflectional**.

3. Root Cause of the Error

The main problem is that the system may treat all prefixes and suffixes as simple removable strings.

This can cause it to:

- Remove meaningful derivational affixes.
- Confuse derivational and inflectional morphemes.
- Lose important medical meaning.
- Produce incorrect roots during indexing.

4. Corrected Strategy

The system should use a morphology-aware analysis:

Input word

↓

Identify prefix
↓
Identify root
↓
Identify suffix
↓
Classify affixes
↓
Preserve morphological information

For example:

untreated
→ un + treat + ed
→ derivational + root + inflectional

5. Comparison

Incorrect approach: Remove affixes without considering their function.

Correct approach: Identify the root and classify every affix as derivational or inflectional.

6. Effect on Biomedical Information Retrieval

Correct morphological analysis helps the search system connect related medical terms without destroying important meaning.

For example:

treat
treatment
treated
treatable

can be recognized as morphologically related forms.

This can improve search matching and reduce incorrect retrieval results.

Result

A morphology-aware approach correctly separates roots, prefixes, and suffixes while distinguishing derivational and inflectional morphemes. This preserves important biomedical information and can improve information retrieval.

Q2. Finite-State Morphological Parsing

The parser has errors with **replayed**, **unhappier**, **disconnected**, **players**, **restarting**, and **unreadable**.

1. Correct Morphological Analysis

Word	Correct decomposition
replayed	re + play + ed
unhappier	un + happy + er
disconnected	dis + connect + ed
players	play + er + s
restarting	re + start + ing
unreadable	un + read + able

2. Main Error

The parser can recognize **one affix**, but fails when a word contains multiple prefixes/suffixes.

For example:

```
replayed  
→ re + play + ed
```

It may recognize `re-` but fail to continue to `-ed`.

3. Correction

The FSA/FST should allow repeated transitions:

```
Prefix → Root → Suffix → Suffix
```

For example:

```
re → play → ed
```

and:

```
un → read → able
```

4. Before vs After

Before:

```
replayed → re + played
```

After:

```
replayed → re + play + ed
```

Similarly:

```
players → play + er + s  
restarting → re + start + ing  
unreadable → un + read + able
```

5. Result

The modified finite-state parser can handle **multiple affixes**, producing more accurate morphological analyses. This improves parsing accuracy, although the rule-based FSA/FST still depends on the affixes and roots included in its design.

Q3. Identify the Error and Correct the Stemming Program

The given program applies `PorterStemmer` directly to the **entire text**, instead of tokenizing the sentences first.

1. Error

This line is wrong:

```
data["Processed"] = data["Text"].apply(ps.stem)
```

`ps.stem()` expects **one word**, not a complete sentence.

2. Correct Program

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("news.csv")

def process_text(text):
    tokens = word_tokenize(text.lower())
    stems = [ps.stem(word) for word in tokens]
    return stems

data["Tokens"] = data["Text"].apply(
    lambda x: word_tokenize(x.lower())
)

data["Stemmed"] = data["Text"].apply(
    process_text
)

print(data[["Text", "Tokens", "Stemmed"]].head())
```

3. Example

Original:

The players are playing games.

Tokens:

```
['the', 'players', 'are', 'playing', 'games']
```

Stemmed tokens:

```
['the', 'player', 'are', 'play', 'game']
```

4. Why This Is Better

The corrected program:

Text


```
↓
Tokenization
↓
Stemming
↓
Stemmed tokens
```

This ensures that each word is processed separately and avoids incorrect stemming of complete sentences.

Result

The main error was applying `ps.stem()` directly to full sentences. Tokenizing the text first and then stemming each token produces the correct morphological preprocessing.

Q4. Finite-State Morphological Parser

The given parser has several logical errors in how it handles plural nouns.

1. Errors in the Original Program

The main error is:

```
word[:-2]
```

It always removes **two characters**, even for regular plurals like `cars`.

Also, it only checks whether a word ends in `s`, so it fails for:

```
boxes
cities
children
```

2. Corrected Program

```
def parser(word):

    irregular = {
        "children": "child"
    }

    if word in irregular:
        return irregular[word], "Plural Noun"

    if word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"

    if word.endswith("es"):
        return word[:-2], "Plural Noun"

    if word.endswith("s"):
        return word[:-1], "Plural Noun"

    return word, "Singular"
```

```
words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
```

```
]

for w in words:
    print(w, "->", parser(w))
```

3. Correct Output

```
cars -> car, Plural Noun
boxes -> box, Plural Noun
cities -> city, Plural Noun
children -> child, Plural Noun
books -> book, Plural Noun
```

4. Rules Used

```
cars    → car
boxes   → box
cities  → city
children → child
books   → book
```

- -s → remove s
- -es → remove es
- -ies → change ies to y
- Irregular plurals → use a dictionary

Result

The corrected parser handles **regular, -es, -ies, and irregular plurals**, unlike the original parser, which blindly chops off two characters like a very enthusiastic lawnmower.

Q5. Stemming Error in Feature Extraction

The given program extracts features **before** applying stemming. The task asks us to correct the preprocessing order.

1. Error

The original order is:

```
Text → Feature Extraction → Stemming
```

This creates separate features such as:

```
connected
connection
connecting
connectivity
```

before normalization.

2. Correct Order

```
Text
↓
Tokenization
↓
Stemming
↓
Feature Extraction
```

3. Corrected Program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
```

```

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

def stem_text(text):
    words = text.lower().split()
    return " ".join(stemmer.stem(w) for w in words)

# Normalize first
normalized = [
    stem_text(doc)
    for doc in documents
]

# Extract features after normalization
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized)

print("Normalized Documents:")
for doc in normalized:
    print(doc)

print("\nFeatures:")
print(vectorizer.get_feature_names_out())

```

4. Example

Before normalization:

```

connected
connection
connecting
connectivity

```

After stemming:

```

connect
connect
connect
connect

```

Similarly:

```

studies
studied
studying
study

```

are normalized before feature extraction.

5. Result

The corrected pipeline reduces redundant morphological forms in the vocabulary. This can make the feature representation more compact and can help an NLP classifier work with more consistent features.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____